

Dynamic Symbolic Execution Tool for Python Programs

Xuefeng Ding¹, Wanyu Huang², Ying Liu³, Chen Wantao³, Ding Xuyang³

¹Sichuan University, Chengdu 610041;

²Center for Cyber Security, University of Electronic Science and Technology of China, Chengdu 611731;

³Power china Chengdu Engineering Corporation Limited, Chengdu 610072

Abstract—Dynamic symbolic execution is an important automated testing technique. Firstly, we introduce the traditional symbolic execution and dynamic symbolic execution technology, and then review the research status of Python application testing with dynamic symbolic execution technology. Then we compare and analyze the dynamic symbolic execution tools with the existing Python programs and Architectures, performance, and supported data types. Finally, the development direction of this field prospects. It is helpful for researchers to understand the progress of dynamic symbolic execution technology in current Python program and lay a foundation for future research work.

Keywords: Dynamic symbolic execution; Automated testing; Python program; Dynamic symbolic execution tool

1 INTRODUCTION

In the field of software development, the development of high accuracy, strong stability and high security of software is extremely difficult [1]. the development process will inevitably introduce software defects or security risks, which threatens the software reliability and safety. Software testing is the most important way to reduce software defects, improve software reliability and security [2]. Latest research shows that software testing accounted for 50% -80% in software development costs [3], so the software testing research has important significance. Traditional software testing uses manual testing methods, which is not only costly but with a high false alarm rate. To save labor costs and reduce the impact of human factors on software testing, software testing needs to move in the direction of automation.

Because of its high coverage rate and low false alarm rate, and the continuous improvement of solving technology, dynamic symbolic execution technology is widely used in automated test case generation, automated filter generation, malware analysis and other fields. At present, the dynamic symbolic execution technique has been successfully applied to the test of static compiling type program, but dynamic symbolic execution tools for dynamic compiling and running programs such as Python are still few. Python language is favored by more and more Web applications, database applications and numerical calculation programs because of its extensibility, embeddability and rich function library support [4]. Therefore, it is of great importance to design and implement a dynamic symbolic implementation tool to test Python applications.

This paper summarizes the features of the existing

tools and gives an objective evaluation of their advantages and disadvantages. Finally, we give the research direction of dynamic symbolic execution tools for Python programs.

2 SYMBOLIC EXECUTION TECHNOLOGY OVERVIEW

2.1 Traditional symbolic execution

The symbolic execution technique was first proposed by James C. King [5] in 1976, the main idea is to use symbol values instead of specific values as program inputs to simulate program execution and to use symbolic expressions to represent program variables associated with input data [6]. When the program branch is encountered, the corresponding path constraint conditions are collected according to the branch-related symbol variables. After the constraints are processed, the constraint solver is used to find the specific values of the symbols that satisfy the path branch. A new test case is generated using the solved value, the program is rerun, and the program execution path coincides with the symbol execution path and terminates in the same way.

As the technology can automatically generate test cases, and in theory to ensure that each test case corresponds to a different test program execution path, greatly reducing the number of useless and redundant test cases, so it plays a major role in the software defect reproduction, automated vulnerability discovery . EGT [7], EXE [8], KLEE [9] are the representative tools. However, traditional symbolic execution techniques can not use specific values during program analysis. Therefore, when the collected path constraints contain criteria that the solver can not solve, it is impossible to generate new test cases. This problem is one of the main obstacles to prevent the development of traditional symbolic execution techniques.

2.2 Dynamic symbolic execution

The key idea of dynamic symbolic execution theory is to use a given or randomly generated initial value to execute the program, but still treat the input as a symbol. The program selects the appropriate branch execution according to the specific input value and collects symbol constraint conditions connected with input at the path branch .Finally, according to the selected path search algorithm, the path constraint is negated and the new input is obtained by using the constraint solver. In principle, the new input value can make the program execute along different paths. The process is then repeated until the program path has been explored or reaches the target coverage, or after the preset test time has elapsed. The technique does not require full-function

symbol interpreters as in the case of traditional symbolic implementations, nor does it require storage state, and can use actual values during execution. Therefore, dynamic symbolic execution theory, which uses dynamic re-execution mode for program analysis, alleviates some of the problems encountered in traditional symbolic execution, such as the call of external functions that are difficult to handle with traditional symbolic execution, complex mathematical calculations and so on, while this can be resolved to a certain extent by the actual value of the program execution with dynamic symbolic execution.

Dynamic symbolic execution technology has high accuracy, high coverage rate, and theoretically zero rates. Therefore, this technology has been paid more attention and focused on the automatic generation of test cases and automated exploits, and in the industrial sector has also produced a certain practical value. Existing dynamic symbolic execution tools are CUTE [10], DART [11], SAGE [12], PEX [13], PathCrawler [14] and so on.

In addition, most of the dynamic symbol execution software were developed by non-profit organizations such as universities and laboratories, and only a handful of software were developed by the company, according to a survey of existing mainstream dynamic symbolic execution tools. Mainly due to the existence of path explosion, external / complex arithmetic operations, floating point calculation, symbol pointer operation, environmental interaction [15] and other issues in this technology. This also shows that the dynamic symbolic execution technology is still in the research stage, more researchers need to conduct the further in-depth study.

3 DYNAMIC SYMBOLIC EXECUTION TOOLS FOR PYTHON CODE

3.1 NICE

3.1.1 Brief introduction

Model detection technology was first proposed in 1981, the technology mainly by explicit state search or implicit fixed-point calculation to verify the finite state of the system with the model / propositional nature. Since model checking can be performed automatically and can provide a counter-example path when the system does not satisfy the nature, it plays an important role in verifying program correctness.

However, there are some problems in using existing well-tested model checking tools such as SPIN and Java Finder (JPF) directly in OpenFlow program, such as: bring great development volume and cause state space explosion. Mainly due to the time constraints of the system abstraction modeling process, and the data dependency problems that exist in OpenFlow applications result in a large number of transition states, input packets, and sequence of events.

In response to these problems, NICE [16] combined with the model detection technology and symbolic execution technology to test the unmodified OpenFlow controller program developed on the NOX platform. NICE can adapt well to the hardware and software capabilities of the NOX platform, thereby maximizing the use of model checking tools on existing general-purpose

computing platforms. As a result, code development workloads are low and the scalability of the model detector can be enhanced with selectable symbolic execution events handler while using an inert initialization technique to mitigate the constraint solver pressure and domain-specific heuristic algorithms to reduce the event sequence space.

3.1.2 Architecture

As can be seen in Figure 1, the input conditions for NICE are the OpenFlow controller program, the network topology, and the correct nature, the output is the path which violates the correct nature. The tool implements the combination of model checking technique and symbolic execution technique in the process of acquiring the next state. The core part of NICE consists of model checking, symbolic execution, which searches the state space of the entire system (controllers, converters, and terminals) and tracks the execution of the Python interpreter.

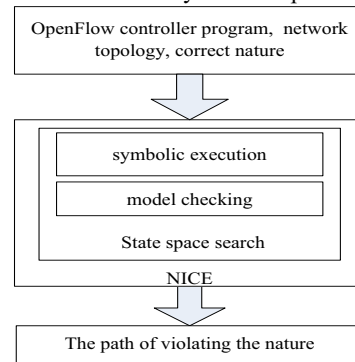


Figure1 NICE Frames

3.2 CuitePy

3.2.1 Brief introduction

The dynamic symbolic execution theory proposed in 2003 has been successfully used to test programs written in static type languages because it can thoroughly test the tested software with the low false positive rate. However, due to the complex semantics, fragment, and incomplete type information of dynamic languages and the call of external functions lacking accurate models, the existing dynamic symbolic execution tools cannot be used directly to test Python programs.

Based on the above features of the Python program, the hybrid execution engine CutiePy [17] based on the formal model of the Python language provides a formal expression that expresses the constraint condition of the path accurately and uses the lazy initialization technique to improve the path search efficiency. In addition, CutiePy uses an uninterpreted ObjectSort to represent a Python object. The subtype of ObjectSort is PyType. PyBool, PyInt, PyNupType, PyTuple, PyList, PyListiter, PyDict, and so on are the constants of Python predefined types. For mutable objects, use the Python bytecode interpreter to introduce additional insights to update references.

3.2.2 Architecture

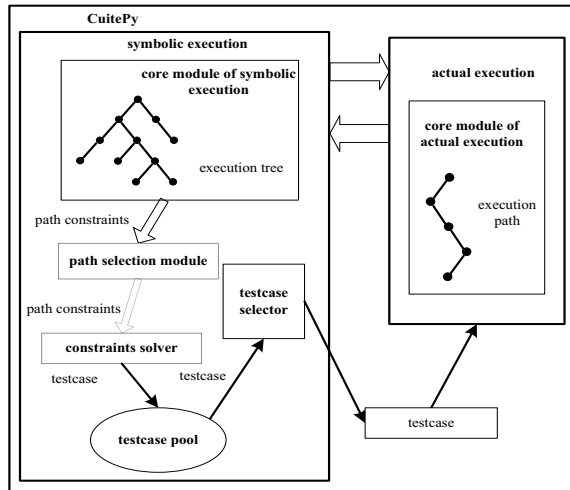


Figure2 CuitePy frames

Figure 2 shows the structure of CuitePy, it can be seen that the tool is a hybrid execution tool. CuitePy first gives the initial test case, uses the custom CPython interpreter to actually run the program under test, updates symbolic expressions of variables according to model semantics, collects path conditions, modifies path conditions, generates new test cases, and then repeats the process. The key of the tool is the constraint representation and collection form of each instruction as well as the branch jump representation mode. It also mitigates the problem that symbolic information cannot be correctly tracking and dissemination during the actual execution by lazy building and solving the initial constraint condition and the complete or partial formalization of the library function.

3.3 Commuter

3.3.1 Brief introduction

Commuter [18] is an automated extensible testing tool that is used to find unnecessary sharing of code. Traditional scalability testing requires a workload that is not required by Commuter, and the tool supports multi-core.

3.3.2 Architecture

From Figure 3, namely the Commuter architecture diagram we found that the tool mainly consists of analysis, testing, validation these three parts. Commuter's input is a high-level software interface model, which uses the symbolic execution engine in the analysis module to compute the exact conditions under the operation set condition, and then uses the test module to generate a specific set of test cases, and finally uses the validation module to expose the execution conflict.

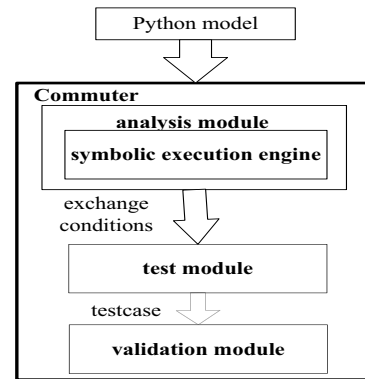


Figure3 Commuter frames

3.4 Chef

3.4.1 Brief introduction

Existing dynamic symbolic executing software automatically testing statically compiled programs has been used quite successfully. Python programs are executed directly by the interpreter and cannot be statically compiled, so these off-the-shelf tools cannot be directly used to symbolize Python programs. Although the NICE engine has made advances in the symbolic implementation of the OpenFlow controller program, it is prone to false positives and low test performance.

The central idea of Chef [19] is to use a language interpreter as an executable specification, that is, to build a symbol execution engine on the interpreter of the interpreted language. Chef's design philosophy not only reduces development effort but also enhances the extensibility of interpretive language symbolic execution. The chef is built on X86's S2E symbolic execution platform, symbolically executes the target program by symbolically executing the interpreter's binary, and uses the hidden knowledge of the high-level structure of the program to reason.

3.4.2 Architecture

Using Chef to test the architecture of Python programs is shown in Figure 4, the Python interpreter and Chef together form the symbolic execution engine for Python programs. The chef is built on the X86 symbolic execution engine S2E and uses the high-level instruction stub to invoke the extended S2E client API, which is invoked by the interpreter to track the high-level path currently being executed. The explored high-level paths are used to build high-level execution trees and low-level to high-level mappings. Chef uses state selection policies to maximize the proportion of high-level and low-level paths.

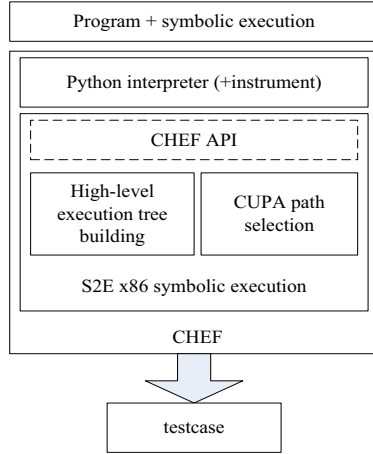


Figure 4 Chef frames for python

3.5 Conpy

3.5.1 Brief introduction

To use the idea of dynamic symbolic execution for Python program detection, Conpy [20] uses an additional property, symbol or symbol_set, to extend a Python object based on the principle that "Python uses everything in the program as an object."

Conpy, which is written in the Python language, can be run as a module in the Python code base, providing a great deal of convenience for Conpy's deployment, allowing Conpy to run on mainstream hardware and software platforms. In addition, Conpy uses selective symbolic execution, without concern for operations that do not involve symbol execution, and the user only needs to specify the input source (ie, data read from the file, data from the Internet, return value of any function) to execute automatically, which provides a good user interaction.

3.5.2 Architecture

From Figure 5, we find that Conpy's dynamic symbolic execution has three procedures: tag symbol source, trace symbol, and generation path conditions. The latter two steps are fully automated, and only the first step requires human interaction. The key to Conpy symbolization is to add properties to Python objects, such as adding the symbol attribute to the original type, adding the symbol_set property to the container, and symbolizing the related items in the collection, making Conpy run on the original type. The basic principle of Conpy tracking symbol propagation is that if a state involves the calculation of a symbol variable, the state is mapped to the target symbol variable. Path constraints are generated during the comparison of branch conditions related to symbolic variables.

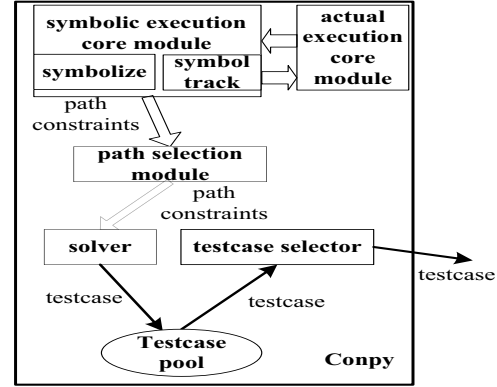


Figure 5 Conpy frames

4 COMPARATIVE ANALYSIS

4.1 Performance analysis

The prerequisite for testing OpenFlow applications with NICE is the need to provide input: Topology specification between controller programs, translators, and end hosts. And NICE identify the equivalent packet class is based on the assumption that all data packets reach the controller, but in fact, the data packets belonging to the same type cannot ensure the same time to reach the converter, this inconsistency reduced the NICE of the test efficiency and code coverage to a certain extent. In addition, when NICE specifying global controller variables, it cannot get the packet header constraints due to the symbolic execution began in the specific global variables. NICE uses reduced input space and Open Flow-specific search algorithm to deal with the infinite execution tree, although to a certain extent, improve efficiency, but the code coverage is reduced.

CuitePy, which update the variable symbolic expression according to the model semantics, cannot ensure that all of the symbolic expression are represented accurately. In addition, the symbol tracking principle of the tool is that the source operand is symbolic and the destination operand should be symbolized. The symbolic principle of CuitePy in the actual implementation process will produce a lot of false positives, such as: $y = x - x$, $y = x / x$, $y = x \text{ (xor) } x$, although x is the symbol value, but the value of y is not relevant to x , its value is certain ($y = x - x = 0$, $y = x / x = 1$ (when x is not equal to 0), $y = x \text{ (xor) } x = 0$). Therefore, CuitePy will symbolize y , wasting a lot of space resources.

Commuter's main role is to solve the delay problem of the software interface multi-core expansion, the use of the tool needs to provide high-level interface model. High-level abstraction modeling of the tested program is time-consuming and complex.

Although Chef has brought a new vision to the symbolic execution of dynamic languages: symbolizing the interpreter and using certain optimization strategies to improve the efficiency of execution, low-level symbolic execution still inevitably faces the path explosion problem. Among them, the high-level symbolic execution refers to the symbolization on the target language,

low-level symbolic execution refers to the symbolization on the interpreter. In a Python program, a high-level execution path may contain multiple low-level execution paths. In the low-level symbolic execution, it needs to execute as many low-level paths as possible, and for the program under test, these low-level paths correspond to the same high-level path, and no increase in branch coverage.

Table 1 performance of dynamic symbolic execution tools for Python programs

tools	technology	Test range	shortcomings
NICE	model checking +symbolic execution	python programs on NOX platform	low test efficiency and low code coverage
CutiePy	actual execution + symbolic execution	Python software package	a waste of space
Commuter	symbolic execution+ formal verification	Python interface	time-consuming and complex high-level modeling
Chef	actual execution+ low-level symbolic execution	Python programs	path explosion problem
Conpy	actual execution +high-level symbolic execution	Python programs	Not support custom python types

Conpy that runs at the byte level can get additional information, for example: to determine whether the two strings are equal, Conpy not only can determine the two strings are not the same can also give two bytes which are different. However, low-level execution will generate a large number of paths, and in the high-level point of view, these paths don't execute a new high-level path, that is not to increase the path coverage, from a certain sense it doesn't bring substantial progress for the verification of test procedure. In addition, in most cases, Conpy's time overhead is at least one order of magnitude lower than existing dynamic symbolic execution tools.

In summary, the performance of the existing dynamic symbolic execution tools NICE, CutiePy, Commuter, Chef, and Conpy are shown in Table 1.

4.2 Support type and test range analysis

Table 2 data types supported by the dynamic symbolic execution tools for Python programs

data types tools	integer	float point	string	container	custom type
NICE	+	-	-	-	*

CutiePy	+	-	-	*	-
Commuter	+	-	*	+	*
Chef	+	-	+	+	+
Conpy	+	-	+	+	-

Based on the research of NICE-PySE, CutiePy, Commuter, Chef and Conpy, and the deep analysis of NICE, Commuter, and Chef, we find that the data types supported by these tools are listed in Table 2 , Where + indicates full support, - indicates no support, and * indicates partial support.

5 CONCLUSION

Symbolic execution is the key technology of software testing. Dynamic symbolic execution frees developers from time-consuming manual inspection program and improves test efficiency. In this paper, the existing symbolic execution techniques are reviewed. Firstly, the existing symbolic execution techniques are divided into traditional symbolic execution and dynamic symbolic execution from the point of concretization. Then, based on the related research results, the paper expatiates on the achievements and shortcomings of the existing symbolic tools for testing Python programs, and analyzes and compares the performance of these tools. Finally, the research trends in this field prospect, and the following research directions are proposed.

1. Research on Searching Effective Path Algorithm

Because the Python program's object type information is incomplete, such as x [1], it can point to lists, tuples, strings, dictionaries, or user-defined types. This makes it difficult to represent and collect path constraints, so a quick search of an efficient path is important to check for software vulnerabilities.

2. Extension of Solver

There are a large number of manipulations of strings and containers in Python programs, and the existing string solvers either ignore the behavior of non-strings in string operations or only support a limited string of operations. There is nothing more than a method of handling string and non-string operation is converted to the same representation, such as bit vector. As an extension of the Z3 solver, Z3-str can solve string operations. Based on this idea, we can extend the container and custom type operations in Python programs so that the solver can effectively support more data types.

3. High-level symbolic execution

Only the existing symbolic execution tool NICE is running on the high-level implementation, and the remaining four are based on low-level symbolic implementation. The NICE is only effective for the development of OpenFlow controller program in the NOX platform, its scalability is not enough. While low-level can get extra information while the program is running, additional information not only requires memory space for storage, it can also cause path explosion problems. Therefore, it is urgent to carry out the research on the symbolic execution of the high-level symbol-based interpretive language to further improve the code coverage of the dynamic symbolic execution tool to test

the interpreted language to maximize the hidden errors and improve the software quality.

REFERENCES

- [1] C. Meudec. ATGen: automatic test data generation using constraint logic programming and symbolic execution [J]. *Software Testing, Verification, and Reliability*, 2001, 11(2): 81-96.
- [2] C. Meudec. ATGen: automatic test data generation using constraint logic programming and symbolic execution [J]. *Software Testing, Verification, and Reliability*, 2001, 11(2):81-96.
- [3] P. Joshi, K. Sen, M. Shlimovich. Predictive testing: amplifying the effectiveness of software testing [R]. Technical Report, 2007, No.UCB/EECS-2007-35.
- [4] W. Chun. *Core Python Programming* (2nd Edition) [M]. London: Prentice Hall PTR, 2006.
- [5] M.J.C. King. Symbolic execution and program testing [J]. *Journal of the ACM*, 1976, 19(7):385-394.
- [6] Cadar C, Sen K. Symbolic execution for software testing: three decades later [J]. *Communications of the ACM*, 2013, 56(2): 82-90.
- [7] C. Cadar, D. Engler. Execution generated test cases: how to make systems code crash itself: proceedings of the 12th International SPIN Workshop on Model checking Software, San Francisco, August 22-24, 2005 [C]. Berlin: Springer, c2005.
- [8] C. Cadar, V. Ganesh, P. Pawlowski, et al. EXE: automatically generating inputs of death: CCS 2006: proceedings of the 13th ACM Conference on Computer and Communications Security, Alexandria, October 30-November 3, 2006 [C]. Berlin: Springer, c2006. Amsterdam: Elsevier, c2006.
- [9] C. Cadar, D. Dunbar, D. Engler. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs: proceedings of the USENIX Symposium on Operating System Design and Implementation, San Diego, December 8-10, 2008. Amsterdam: Elsevier, c2008.
- [10] K. Sen, D. Marinov, G. Agha. CUTE: a concolic unit testing engine for C: proceedings of the 13th ACM SIGSOFT Symposium on the Foundations of Software Engineering, Lisbon, September 5-9, 2005[C]. Amsterdam: Elsevier, c2005. London: IET, c2005.
- [11] P. Godefroid, N. Klarlund, K. Sen. DART: directed automated random testing: proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, Haifa, October 19-22, 2009[C]. Berlin: Springer, c2009.
- [12] P. Godefroid, M. Levin, D. Molnar. Automated whitebox fuzz testing: proceedings of the Network and Distributed System Security Symposium, San Diego, February 8-11, 2008[C]. Amsterdam: Elsevier, c2008.
- [13] N. Tillmann and J. De Halleux. Pex-white box test generation for .NET: proceedings of the 2nd International Conference on Tests and Proofs, April 9-11, 2008[C]. Berlin: Springer, c2008.
- [14] N. Williams, B. Marre, P. Mouy, et al. PathCrawler: automatic generation of path tests by combining static and dynamic analysis: proceedings of the 5th European Dependable Computing Conference, April 20-22, 2005[C]. Berlin: Springer, c2005.
- [15] Chen T, Zhang X, Guo S, et al. State of the art: Dynamic symbolic execution for automated test generation[J]. *Future Generation Computer Systems*, 2013, 29(7): 1758-1773.
- [16] Canini M, Venzano D, Peresini P, et al. A NICE Way to Test OpenFlow Applications: Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation, San Jose, April 25-27, 2012[C].
- [17] Sapra S, Minea M, Chaki S, et al. Finding errors in python programs using dynamic symbolic execution[M]. *Testing Software and Systems*. Springer Berlin Heidelberg, 2013: 283-289.
- [18] Clements A T, Kaashoek M F, Zeldovich N, et al. The scalable commutativity rule: Designing scalable software for multicore processors [J]. *ACM Transactions on Computer Systems (TOCS)*, 2015, 32(4): 10.
- [19] Bucur S, Kinder J, Candea G. Prototyping symbolic execution engines for interpreted languages: proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems, Salt Lake City, March 1-5, 2014[C]. Amsterdam: Elsevier, c2014.
- [20] Chen T, Zhang X, Chen R, et al. Conpy: Concolic Execution Engine for Python Applications [M]. *Algorithms and Architectures for Parallel Processing*. Springer International Publishing, 2014: 150-16